HDOC Day-58 Activity

Kth Smallest Element Using Bubble Sort and Pointers

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-58

Aim

To write a C program that sorts a variable-length integer array in ascending order using the **bubble sort algorithm** and displays the kth smallest element. The array is passed to **kth_smallest()** using its address and accessed using pointers.

Problem Requirements

- Read the size and elements of **arr[]** and an integer **k**.
- Pass the array to **kth_smallest()** using its address.
- Sort the array in ascending order using bubble sort.
- Display the element at the kth position of the sorted array.
- Count duplicate values as separate elements.
- Display **-1** when k is invalid.

Algorithm

1. Read n, the n array elements, and k.
2. Pass arr, n, and k to **kth_smallest(arr, n, k)**.
3. If k <= 0 or k > n, print -1 and return.
4. Apply bubble sort. Compare each adjacent pair using pointer notation.
5. If the left element is greater than the right element, swap them.
6. After sorting, the kth smallest element is stored at index **k - 1** because array indexing starts from 0.
7. Display ***(arr + k - 1)**.

C Program

```
#include <stdio.h>

void kth_smallest(int *arr, int n, int k)
{
    if (k <= 0 || k > n)
    {
        printf("-1");
        return;
    }

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (*(arr + j) > *(arr + j + 1))
            {
                int temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }

    printf("kth smallest element: %d", *(arr + k - 1));
}

int main()
{
    int n, k;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    kth_smallest(arr, n, k);

    return 0;
}
```

Pointer Usage

kth_smallest(arr, n, k) passes the address of the first element. The parameter **int *arr** receives that address. Expressions such as ***(arr + j)** and ***(arr + j + 1)** access adjacent elements during bubble sort.

Bubble Sort Dry Run - Example 1

Stage	Array
Original	7 10 4 3 20 15
After Pass 1	7 4 3 10 15 20
After Pass 2	4 3 7 10 15 20

After Pass 3	3 4 7 10 15 20
Sorted	3 4 7 10 15 20

For k = 3, index k-1 = 2. The element at index 2 in the sorted array is **7**.

Test Cases / Expected Results

Case	Input Array	k	Expected Output
Example 1	7 10 4 3 20 15	3	kth smallest element: 7
Example 2	7 10 4 3 20 15	4	kth smallest element: 10
Example 3	7 10 4 3 20 15	7	-1
Example 4	4 2 2 7 5	3	kth smallest element: 4

Duplicate Elements - Example 4

The array **4 2 2 7 5** becomes **2 2 4 5 7** after sorting. Duplicate values are not removed. They occupy separate positions, so the 1st element is 2, the 2nd element is another 2, and the 3rd element is **4**.

Invalid k

If k is less than or equal to 0 or greater than n, there is no valid kth position. Therefore, the function displays **-1**.

Complexity

Time Complexity: $O(n^2)$ because bubble sort uses nested loops.

Space Complexity: O(1) extra space because sorting is performed directly in the input array.

Result

The program successfully sorts the array in ascending order using bubble sort and pointer-based access, then displays the kth smallest element. Duplicate values are counted separately and invalid k values produce -1.

Execution Example

```
Enter the size of array: 6
Enter the elements: 7 10 4 3 20 15
Enter k: 3
kth smallest element: 7
```